Technical Documentation

1. Introduction

This document provides a detailed explanation of the RAG-based Customer Support Assistant system.

It describes the design, implementation, and working of the system, focusing on how it processes documents and generates accurate responses to user queries.

- **What is RAG (Retrieval-Augmented Generation)**

  Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval and text generation.

  The system first retrieves relevant data from a knowledge base and then uses a language model to generate a response based on that data.

- **Why RAG is Needed**

  Traditional language models may generate incorrect or irrelevant answers.

  RAG improves accuracy by grounding responses in actual data from documents, reducing hallucination and increasing reliability.

- **Use Case Overview**

  The system is designed for customer support applications.

  It allows users to ask questions related to policies or documents and receive accurate answers without manually searching through the content.

2. System Architecture Explanation

The system architecture is designed based on a Retrieval-Augmented Generation (RAG) pipeline.

It integrates document processing, vector storage, retrieval, and language model generation to provide accurate responses.

- **Architecture Overview**

  The system consists of multiple layers including document ingestion, embedding generation, vector database storage, retrieval mechanism, and LLM processing.

  These layers work together to transform raw document data into meaningful answers.

- **Component Interactions**

  1. The PDF document is loaded and processed into smaller chunks.

2. Each chunk is converted into embeddings and stored in ChromaDB.
3. When a user enters a query, it is converted into embeddings.
4. The retriever searches the vector database for relevant chunks.
5. The retrieved context is passed to the LLM.
6. The LLM generates a response based on the provided context.
7. If required, the response is escalated to human support.

3. Design Decisions

Several design decisions were made to ensure the system performs efficiently and provides accurate results. These decisions focus on chunking, embeddings, retrieval, and prompt design.

- **Chunk Size Choice**

  A chunk size of around 1000 characters with overlap is used.

  This helps maintain context while ensuring the chunks are not too large for processing.

- **Embedding Strategy**

  HuggingFace embeddings (all-MiniLM-L6-v2) are used because they are lightweight, fast, and free to use.

  They provide good semantic understanding for similarity search.

- **Retrieval Approach**

  Similarity-based retrieval is used to find relevant chunks from the vector database.

  This ensures that the most contextually relevant information is selected for answering queries.

- **Prompt Design Logic**

  The prompt is designed to include both the retrieved context and the user query.

  This ensures that the LLM generates answers strictly based on the provided data, reducing hallucination.

4. Workflow Explanation

The system workflow is designed using LangGraph to manage the flow of data between different components.

It ensures structured execution from query input to final output.

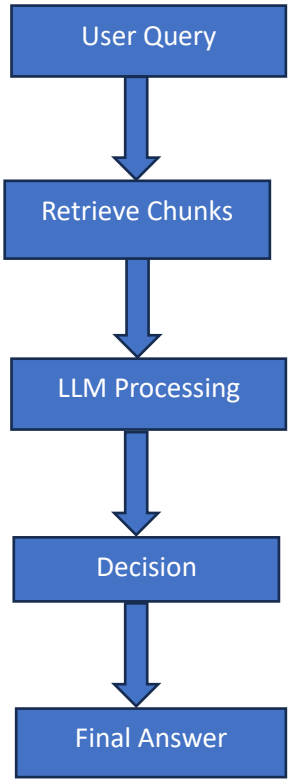

Figure: Workflow of RAG-Based Customer Support Assistant

- ## LangGraph Usage

LangGraph is used to define nodes and edges that control how data moves through the system.

It helps implement a structured pipeline for processing queries and generating responses.

- ## Node Responsibilities

1. Processing Node:

Handles retrieval of relevant chunks and generates a response using the LLM.

2. Output Node:

Determines the final output, either returning the generated answer or escalating the query.

- ## State Transitions

The state object carries data such as query, response, and final output between nodes.

Each node updates the state, and the updated information is passed to the next node in the workflow.

5. Conditional Logic

The system uses conditional logic to decide how to respond to user queries.

Based on different conditions, the system either generates an answer or escalates the query to human support.

- **Intent Detection**

  The system identifies the intent of the query based on its relevance to the document.

  If the query matches the document context, it proceeds with answer generation.

- **Routing Decisions**

  If relevant information is found and a meaningful response is generated, the system returns the answer.

  If no relevant data is found or the response is insufficient, the query is routed to human support.

6. HITL Implementation

The Human-in-the-Loop (HITL) implementation allows the system to involve human support when the AI cannot provide a reliable answer.

This improves accuracy and ensures better user experience.

- **Role of Human Intervention**

  Human intervention is required when the system fails to generate a confident or meaningful response.

  A human expert can review the query and provide the correct answer.

- **Benefits and Limitations**

  Benefits:

  1. Improves response accuracy.
  2. Handles complex queries effectively.
  3. Builds user trust.

  Limitations:

  1. Increases response time.
  2. Requires human resources.

3. Not fully automated.

7. Challenges & Trade-offs

While designing the RAG-based system, several challenges and trade-offs were considered to balance performance, accuracy, and efficiency.

- **Retrieval Accuracy vs Speed**

  Improving retrieval accuracy may require searching more data, which can increase response time.

  A balance is maintained to ensure both speed and accuracy.

- **Chunk Size vs Context Quality**

  Larger chunks provide better context but increase processing time.

  Smaller chunks improve speed but may lose context.

  A moderate chunk size is chosen to balance both.

- **Cost vs Performance**

  Using high-performance models improves accuracy but may increase cost.

  Free and efficient models are selected to maintain a balance between performance and cost.

8. Testing Strategy

The system is tested to ensure that it retrieves correct information and generates accurate responses for different types of queries.

- **Testing Approach**

  The system is tested using various queries related to the PDF document.

  Both simple and complex queries are used to evaluate performance.

- **Sample Queries**

  1. What is the attendance policy?

  2. What are the safety rules?

  3. What is the purpose of this document?

9. Future Enhancements

The system can be further improved by adding additional features and capabilities to enhance performance and usability.

- **Multi-Document Support**

The system can be extended to handle multiple PDF documents, allowing users to query across different data sources.

- **Feedback Loop**

User feedback can be used to improve the system over time by updating the knowledge base and refining responses.

- **Memory Integration**

The system can include memory to store previous interactions, enabling more personalized and context-aware responses.

- **Deployment**

The system can be deployed as a web application using frameworks like Streamlit or FastAPI for real-time user interaction.